Sarathi AI Labs

Logo

Playwright Training Guide

Comprehensive Automation Training — Beginner to Advanced

Prepared by: Sarathi AI Labs

Version: 1.0 | Date: April 2026

Confidential — For Internal Use Only

Module 1: Introduction to Playwright

Playwright is a modern end-to-end testing framework built by Microsoft. It supports **Chromium**, **Firefox**, and **WebKit** — enabling cross-browser testing with a single API.

Why Playwright?

- **Auto-waiting**: No need for explicit waits — Playwright waits for elements automatically
- **Multi-browser**: Test on Chromium, Firefox, and WebKit with one codebase
- **Network Interception**: Mock and modify network requests easily
- **Trace Viewer**: Built-in debugging tool for test analysis
- **Parallel Execution**: Run tests concurrently for faster feedback

Prerequisites

Tool	Version	Purpose
Python	3.10+	Programming language
Playwright	Latest	Test framework
pytest	7.x+	Test runner
VS Code	Latest	Code editor

Module 2: Setting Up Your Environment

Installation

```
pip install playwright pytest-playwright
playwright install
```

 © Sarathi AI Labs

Project Structure

```
project/
├── pages/
│   ├── home_page.py
│   ├── login_page.py
│   └── product_page.py
├── tests/
│   ├── ui/
│   │   ├── test_login.py
│   │   └── test_search.py
│   └── api/
│       └── test_users.py
├── utils/
│   └── helpers.py
├── conftest.py
└── pytest.ini
```

Module 3: Writing Your First Test

```python
from playwright.sync_api import Page, expect

def test_homepage_title(page: Page):
    """Verify the homepage loads with correct title."""
    page.goto("https://example.com")
    expect(page).to_have_title("Example Domain")
```

Key Concepts

1. **Page Object**: Represents a browser tab
2. **Locators**: Find elements on the page
3. **Assertions**: Validate expected behavior
4. **Fixtures**: Shared test setup/teardown

> **Pro Tip:** Always use `expect()` assertions over manual checks. They include auto-retry logic and produce better error messages.

 © Sarathi AI Labs

Module 4: Advanced Locators

```
# CSS Selectors
page.locator(".product-card")

# Text-based
page.get_by_text("Add to Cart")

# Role-based (recommended)
page.get_by_role("button", name="Submit")

# Test ID (most stable)
page.get_by_test_id("login-button")
```

Locator Best Practices

- ✅ Prefer `get_by_role()` and `get_by_test_id()`
- ✅ Use `get_by_text()` for visible labels
- ❌ Avoid XPath unless absolutely necessary
- ❌ Don't use fragile CSS selectors tied to styling

Module 5: Page Object Model (POM)

```
class LoginPage:
    def __init__(self, page: Page):
        self.page = page
        self.email = page.get_by_label("Email")
        self.password = page.get_by_label("Password")
        self.submit = page.get_by_role("button", name="Sign In")

    def login(self, email: str, password: str):
        self.email.fill(email)
        self.password.fill(password)
        self.submit.click()
```

Document generated by Sarathi AI Labs PDF Builder

 © Sarathi AI Labs